TypeScript a través de client.ts

Clase 01 · De los conceptos básicos al código real de `src/api/client.ts`

Cómo está organizada esta clase

Cada tema sigue el mismo recorrido: **1)** el concepto con palabras, **2)** un ejemplo sencillo sin nada del proyecto, **3)** un recuadro amarillo *En client.ts* que señala dónde aparece en el archivo real y por qué se usó ahí.

Las partes van de menos a más: primero lo que TypeScript *es*, después JavaScript moderno que aparece mucho, luego los tipos que "razonan" (narrowing, predicados), después genéricos y utility types, y al final un recorrido completo del archivo usando todo lo anterior.

Al final tienes una chuleta (sintaxis → significado → línea) y ejercicios con solución.

Índice

Parte I · Fundamentos — 1. Tipos que desaparecen · 2. `type`, literales y uniones · 3. Objetos e `interface` · 4. Acceso indexado `T['clave']`

Parte II · JavaScript moderno — 5. Desestructuración, rest y spread · 6. Clases y herencia de `Error` · 7. Promesas, `async`/`await`, `try`/`catch`

Parte III · Tipos que razonan — 8. `any` frente a `unknown` · 9. Narrowing · 10. Predicados de tipo `x is T` · 11. Aserciones `as`

Parte IV · Tipos que fabrican tipos — 12. Genéricos `<T>` · 13. `Record`, `Pick`, `Omit`, `Extract` · 14. `never` y el truco `AssertNoOverlap`

Parte V · Recorrido completo de client.ts — 15. El archivo de arriba abajo · 16. Los cinco finales posibles de `apiFetch`

Anexos — Chuleta · Ejercicios · Soluciones

Parte I · Fundamentos

1. TypeScript: JavaScript con tipos que desaparecen

TypeScript es JavaScript al que le añades **anotaciones de tipo**: le dices al compilador qué clase de valor esperas en cada sitio, y él te avisa *antes de ejecutar* si algo no encaja. La sintaxis básica es `nombre: tipo`.

```
let edad: number = 30
let nombre: string = 'Ana'

function sumar(a: number, b: number): number {   // parámetros y retorno
  return a + b
}

sumar(2, 3)      // ok
sumar(2, '3')    // ✗ error de compilación: '3' no es number
```

La idea más importante de toda la clase: **el navegador nunca ve los tipos**. Antes de ejecutar, Vite (y `vue-tsc` al comprobar) borra todas las anotaciones. Lo que llega al navegador es JavaScript normal:

```
// Lo que tú escribes (TS)              // Lo que se ejecuta (JS)
let edad: number = 30                   let edad = 30
type Color = 'rojo' | 'verde'           // (desaparece entero)
function sumar(a: number, b: number) {  function sumar(a, b) {
```

Idea clave: dos mundos

Hay un **mundo de los tipos** (solo existe mientras compilas: `type`, `interface`, anotaciones) y un **mundo de los valores** (existe al ejecutar: `const`, funciones, clases, arrays).

Consecuencia práctica: en tiempo de ejecución *no puedes preguntar* "¿este string pertenece al tipo X?", porque X ya no existe. Si necesitas comprobarlo de verdad, necesitas un **valor** (por ejemplo un array con las opciones válidas).

Las clases son especiales: son a la vez tipo y valor (por eso `instanceof` funciona).

En client.ts

La primera línea de código ya mezcla los dos mundos:

```
import { apiErrorCodeValues, type components } from '@/types/backend'
```

`apiErrorCodeValues` es un **valor**: un array real con los códigos de error del back (`['authentication_required', 'email_already_registered', …]`). `components` es solo un **tipo** y la palabra `type` delante lo marca, para que al compilar ese import se borre. Ambos vienen de `src/types/backend.ts`, que genera `npm run generate:types` desde el OpenAPI del back.

El comentario de cabecera del archivo explica justo esto: el tipo `ApiErrorCode` es "cerrado" en el compilador, pero el back puede enviar un código nuevo en runtime, así que se comprueba contra el array `apiErrorCodeValues` (sección 10).

2. Alias con type, tipos literales y uniones

`type` le pone **nombre** a un tipo para reutilizarlo. No crea nada en runtime; es un apodo.

```
type Id = number
type Nombre = string
let usuario: Id = 42
```

Un **tipo literal** es un tipo que solo admite *un valor exacto*. Solos no sirven de mucho, pero combinados con `|` (**unión**, se lee "o") forman listas cerradas de opciones:

```
type Semaforo = 'rojo' | 'ambar' | 'verde'

let luz: Semaforo = 'rojo'    // ok
luz = 'azul'                  // ✗ '"azul"' no es asignable a 'Semaforo'
```

Las uniones se pueden combinar entre sí; el resultado es la unión de todos los miembros:

```
type Frio = 'invierno' | 'otoño'
type Calor = 'verano' | 'primavera'
type Estacion = Frio | Calor        // los cuatro valores
```

Cuando la unión es larga se suele escribir un `|` delante de cada línea, incluido el primero. Ese `|` inicial es solo formato, no significa nada extra.

En client.ts

```
type FrontOnlyApiErrorCode = 'unrecognized' | 'network_error'

export type ApiErrorCode =
  | ContractApiErrorCode        // los 10 códigos que define el back
  | FrontOnlyApiErrorCode       // 2 códigos que solo inventa el front
  | AssertNoOverlap<...>        // truco de la sección 14 (aporta 0 miembros)
```

`ApiErrorCode` es la lista completa de códigos que verá el resto de la app: los del contrato del back más dos propios del front (`unrecognized` cuando el back responde algo que no entendemos, `network_error` cuando ni siquiera hay respuesta). Así quien captura un error siempre recibe **un solo tipo** con un `code` de una lista cerrada, y el editor te autocompleta los valores.

`export` delante de `type` lo hace importable desde otros archivos.

3. Tipos de objeto e interface

Para describir la **forma** de un objeto (qué propiedades tiene y de qué tipo) se usa `interface` (o `type` con llaves; en la práctica son casi equivalentes).

```
interface Usuario {
  readonly id: number      // readonly: no se puede reasignar después
  nombre: string
  email?: string           // ?: opcional → su tipo es string | undefined
}

const u: Usuario = { id: 1, nombre: 'Ana' }   // ok, email puede faltar
u.nombre = 'Eva'                              // ok
u.id = 2                                      // ✗ id es de solo lectura
```

Una interface puede **extender** otra y heredar sus propiedades:

```
interface Admin extends Usuario {
  permisos: string[]
}
// Admin = { readonly id, nombre, email?, permisos }
```

Ojo con readonly

`readonly` es una regla del **compilador**: en runtime el objeto es un objeto normal. Sirve para expresar intención ("esto no debería cambiar") y que TypeScript te frene si lo intentas.

En client.ts

```
interface ApiErrorInit {
  code: ApiErrorCode
  message: ContractApiError['message']   // = string (sección 4)
  status: number
  unrecognizedCode?: string                  // opcional
}
```

Es la forma del objeto que recibe el constructor de `ApiError`. `unrecognizedCode` es opcional porque solo tiene sentido cuando el código del back no se reconoce. En la clase, las propiedades llevan `readonly`: un error, una vez creado, no debería modificarse.

4. Acceso indexado: T['clave']

Igual que en JavaScript haces `objeto['clave']` para sacar un **valor**, en TypeScript puedes hacer `Tipo['clave']` para sacar el **tipo** de esa propiedad. Y se puede encadenar.

```
interface Pedido {
  id: number
  cliente: { nombre: string; ciudad: string }
}

type IdPedido = Pedido['id']                    // number
type Cliente  = Pedido['cliente']               // { nombre: string; ciudad: string }
type Ciudad   = Pedido['cliente']['ciudad']     // string
```

Sirve para **no repetir**: si mañana `id` pasa a ser `string`, `IdPedido` cambia solo.

En client.ts

El archivo generado `src/types/backend.ts` mete todos los esquemas del back dentro de un gran tipo `components`. Simplificado, se parece a esto:

```
interface components {
  schemas: {
    ApiErrorCode: 'authentication_required' | 'email_already_registered' | … (10)
    ApiError: { code: components['schemas']['ApiErrorCode']; message: string }
    ApiErrorValidation: { code: …; message: string;
                          violations: { [key: string]: string[] } }
    …
  }
}
```

Y client.ts saca de ahí lo que necesita:

```
type ContractApiError           = components['schemas']['ApiError']
type ContractApiErrorValidation = components['schemas']['ApiErrorValidation']
type ContractApiErrorCode       = ContractApiError['code']    // unión de 10 códigos
```

¿Por qué el prefijo `Contract`? Porque client.ts define sus propias *clases* `ApiError` y `ApiErrorValidation`. Si el alias se llamara igual habría choque de nombres. `Contract…` = "la forma que dice el contrato del back"; sin prefijo = "la clase de error del front".

Parte II · JavaScript moderno que aparece mucho

Esta parte no es TypeScript: es JavaScript (ES2018+). Pero aparece tanto en client.ts que sin ella el resto no se lee.

5. Desestructuración, rest y spread

Desestructurar: sacar propiedades a variables

```
const persona = { nombre: 'Ana', edad: 30, ciudad: 'Bilbao' }

const { nombre, edad } = persona      // nombre = 'Ana', edad = 30
// equivale a:
// const nombre = persona.nombre
// const edad = persona.edad
```

Rest (...) al desestructurar: "el resto"

```
const { nombre, ...resto } = persona
// nombre = 'Ana'
// resto  = { edad: 30, ciudad: 'Bilbao' }   ← todo lo que no he nombrado
```

Spread (...) al construir: "copia esto aquí dentro"

```
const base = { color: 'rojo', talla: 'M' }
const a = { ...base, talla: 'L' }   // { color: 'rojo', talla: 'L' }
const b = { talla: 'L', ...base }   // { talla: 'M', color: 'rojo' }
// ¡El orden importa! Lo que va DESPUÉS sobrescribe lo de antes.
```

Desestructurar en los parámetros y valores por defecto

```
function saludar({ nombre, edad }: { nombre: string; edad: number }) {
  return `${nombre} tiene ${edad}`
}
saludar({ edad: 30, nombre: 'Ana' })   // el orden de las claves da igual

function conOpciones(opciones: { verbose?: boolean } = {}) { … }
conOpciones()                          // opciones vale {} si no pasas nada
```

> **Mismo símbolo, dos papeles**
>
> `...` a la **izquierda** del `=` (o en un parámetro) **recoge** lo que sobra: es *rest*.
>
> `...` dentro de un objeto que estás **creando** lo **esparce**: es *spread*.

En client.ts

Aparece en casi cada bloque:

```
constructor({ code, message, status, unrecognizedCode }: ApiErrorInit)    // l.47
constructor({ violations, ...init }: ApiErrorValidationInit)          // l.63
super({ ...init, code: 'validation_failed' })                         // l.64
const { status } = response                                           // l.92
const { body, headers, ...rest } = options                            // l.129
const { data } = (await response.json()) as { data: T }               // l.153
```

Línea 63-64: se separa `violations` (lo usa la subclase) del resto (`message`, `status`), y ese resto se reenvía al constructor padre añadiendo `code: 'validation_failed'`.

Línea 129: `body` y `headers` se sacan aparte porque hay que tratarlos (convertir a JSON, mezclar cabeceras); `rest` (`method`, `signal`, ...) se pasa tal cual a `fetch` con `...rest`.

Líneas 135-140: las cabeceras por defecto van primero y `...headers` del llamante después, así quien llama puede sobrescribir `Content-Type` si lo necesita (el orden importa).

Línea 127: `options: RequestOptions = {}` — si no pasas opciones, vale un objeto vacío y la desestructuración de la línea 129 no explota.

¿Por qué el constructor recibe un objeto y no cuatro argumentos sueltos? Con `new ApiError('x', 'msg', 400, undefined)` hay que recordar el orden y rellenar huecos; con `new ApiError({ code: 'x', message: 'msg', status: 400 })` cada valor va con su nombre, el orden da igual y lo opcional simplemente no se escribe.

6. Clases y herencia (extends Error)

Una **clase** es una plantilla para crear objetos con propiedades y métodos. `extends` hereda de otra clase, y en el constructor de la hija hay que llamar a `super(...)` (el constructor del padre) **antes** de usar `this`.

```
class Animal {
  readonly nombre: string               // declarar la propiedad y su tipo
  constructor(nombre: string) {
    this.nombre = nombre
  }
}

class Perro extends Animal {
  readonly raza: string
  constructor(nombre: string, raza: string) {
    super(nombre)                       // primero el padre
    this.raza = raza                    // luego lo propio
  }
}

const p = new Perro('Toby', 'beagle')
p instanceof Perro    // true
p instanceof Animal   // true  ← un Perro también ES un Animal
```

Heredar de Error

`Error` es la clase de error nativa de JavaScript. Heredar de ella te da gratis `message` y la traza (`stack`), y hace que tu error se comporte como cualquier otro en `throw`/`catch`. Por convención se cambia `this.name`, que es lo que la consola muestra delante del mensaje (`ApiError: El email ya existe` en vez de `Error: …`).

En client.ts

```
export class ApiError extends Error { … }                 // l.41
export class ApiErrorValidation extends ApiError { … }    // l.60
```

Jerarquía: `Error` → `ApiError` → `ApiErrorValidation`. Un error de validación **también es** un `ApiError`, así que quien llama puede elegir el nivel de detalle:

```
try {
  await register(payload)
} catch (e) {
  if (e instanceof ApiErrorValidation) {
    // e.violations → { email: ['Ya está registrado'], password: [...] }
  } else if (e instanceof ApiError) {
    // e.code → 'email_already_registered' | 'network_error' | ...
  } else {
    throw e   // algo inesperado: que siga subiendo
  }
}
```

La subclase añade lo único que tiene de más (`violations`) y **fija** su código: en la línea 64 siempre pasa `code: 'validation_failed'`, así no se puede crear un `ApiErrorValidation` con otro código.

7. Promesas, async/await, try/catch y throw

Una petición HTTP tarda. `fetch` no devuelve la respuesta, devuelve una **Promise**: "te daré una `Response` más tarde (o fallaré)". En TypeScript el tipo se escribe `Promise<Response>`.

```
async function cargar(): Promise<string> {   // async ⇒ siempre devuelve una Promise
  const res = await fetch('/hola')            // await: espera y "desenvuelve" la Promise
  return await res.text()
}

try {
  const texto = await cargar()
} catch (e) {                                  // si algo falla dentro, se salta aquí
  console.error(e)
}
```

Dos formas de capturar fallos de una promesa: `try`/`catch` alrededor de `await`, o el método `.catch(fn)` encadenado, que convierte el fallo en el valor que devuelva `fn`:

```
const datos = await res.json().catch(() => null)
// si el cuerpo no es JSON válido, datos = null en vez de lanzar
```

En client.ts

Línea 93: `await response.json().catch(() => null)` — si el back (o un proxy) devolvió HTML en vez de JSON, no revienta: `body` vale `null` y luego se trata como "respuesta sin sobre".

Líneas 131-147: `let response: Response` se declara *fuera* del `try` para poder usarla después. TypeScript sabe que tras el `try/catch` está asignada porque el `catch` **siempre** lanza (`throw`): si llegamos a la línea 149, el `fetch` fue bien.

Línea 150: `throw await toApiError(response, path)` — `toApiError` es `async` (devuelve `Promise<ApiError>`); el `await` saca el `ApiError` de la promesa y el `throw` lo lanza.

Parte III · Tipos que razonan

8. any frente a unknown

Ambos significan "puede ser cualquier cosa", pero se comportan al revés:

```
let a: any = JSON.parse(texto)
a.lo.que.quieras()         // TypeScript no se queja… y en runtime puede explotar

let u: unknown = JSON.parse(texto)
u.nombre                   // ✗ error: 'u' is of type 'unknown'
if (typeof u === 'string') {
  u.toUpperCase()          // ok: aquí dentro TS ya sabe que es string
}
```

Regla mental

`any` = "apaga el comprobador". `unknown` = "no sé qué es, **oblígame** a comprobarlo antes de usarlo".

Para datos que vienen de fuera (red, JSON, `localStorage`), `unknown` es lo honesto. Por eso AGENTS.md pide no usar `any`.

En client.ts

Línea 93: `const body: unknown = await response.json()…` — `response.json()` está tipado como `Promise<any>`. Anotar `body` como `unknown` corta ese `any` en seco: a partir de aquí, cada propiedad que leamos hay que comprobarla primero (secciones 9 y 10).

Línea 71: `body?: unknown` en las opciones — quien llama puede pasar cualquier cosa como cuerpo; `apiFetch` no lo inspecciona, solo lo pasa a `JSON.stringify`.

Línea 143: en `catch (error)`, `error` es `unknown` (con la configuración estricta del proyecto): en JavaScript se puede lanzar *cualquier* cosa, no solo `Error`.

9. Narrowing (estrechamiento de tipos)

Narrowing es cuando TypeScript, al ver una comprobación tuya (`if`, `&&`, `?:`), deduce un tipo más concreto dentro de esa rama. No tienes que decirle nada: lee tu código.

```
function describir(x: string | number | null) {
  if (x === null) return 'nada'            // aquí x: null
  if (typeof x === 'string') {
    return x.toUpperCase()                 // aquí x: string
  }
  return x.toFixed(2)                      // aquí x: number (lo único que queda)
}
```

Las comprobaciones que TypeScript entiende y que salen en client.ts:

Comprobación	Qué deduce	Ejemplo en client.ts
`typeof x === 'string'`	x es string	l.79, l.96, l.119
`typeof x === 'object' && x !== null`	x es un objeto (no null)	l.75 (isRecord)

Comprobación	Qué deduce	Ejemplo en client.ts
`x instanceof Clase`	x es una instancia de esa clase	l.145 (TypeError)
`Array.isArray(x)`	x es un array	l.86
`x !== undefined`	quita undefined del tipo	l.141
`a && b`	en b ya vale lo deducido en a	l.84, l.96
`if (x !== 'a') return …`	después del if, x solo puede ser 'a' (por descarte)	l.98-102

Narrowing por descarte: después de un return

TypeScript también razona sobre **lo que ya no puede ser**. Si una rama termina con `return` (o `throw`), el código de después solo se ejecuta cuando la condición fue falsa, y el tipo se estrecha en consecuencia:

```
type Estado = 'ok' | 'error' | 'pendiente'

function procesar(e: Estado) {
  if (e !== 'error') {
    return 'todo bien'     // aquí e: 'ok' | 'pendiente'
  }
  return e                 // aquí e: 'error' (lo único que queda)
}
```

La trampa histórica de typeof null

En JavaScript `typeof null === 'object'` (un error de diseño de 1995 que nunca se pudo arreglar). Por eso para saber si algo es un objeto de verdad hacen falta **las dos** comprobaciones: `typeof value === 'object' && value !== null`. Es exactamente la línea 75.

En client.ts

Línea 145: `if (!(error instanceof TypeError)) throw error` — `fetch` falla con `TypeError` cuando no hay respuesta (sin red, servidor caído, CORS). Cualquier otra cosa (por ejemplo una cancelación con `AbortController`, que lanza otro tipo) no es un fallo de red y se relanza tal cual.

Línea 119: `typeof error.code === 'string' ? error.code : undefined` — guardamos el código recibido solo si de verdad es un string.

10. Predicados de tipo: value is T

El narrowing funciona solo con comprobaciones que TypeScript conoce. Si metes tu comprobación en una función, TypeScript solo ve que devuelve `boolean` y *pierde* la deducción:

```
function esTexto(x: unknown): boolean {
  return typeof x === 'string'
}
const v: unknown = 'hola'
if (esTexto(v)) {
  v.toUpperCase()     // ✗ v sigue siendo unknown: TS no sabe qué significa 'true'
}
```

Un **predicado de tipo** cambia el tipo de retorno a `x is T`: "si devuelvo `true`, entonces `x` es T". Ahora la función se comporta como un `typeof` hecho a medida:

```
function esTexto(x: unknown): x is string {
  return typeof x === 'string'
}
if (esTexto(v)) {
  v.toUpperCase()     // ok: v es string aquí dentro
}
```

Es una promesa que tú haces

TypeScript **no verifica** que tu función compruebe lo que dice el predicado: se fía. Si escribes `x is string` pero compruebas otra cosa, los tipos mentirán. Por eso estas funciones deben ser pequeñas y obvias, como las tres de client.ts.

En client.ts

Hay tres "guardias" (líneas 74-89):

```
isRecord(value: unknown): value is Record<string, unknown>
    → ¿es un objeto (no null)? Entonces puedo leer value.loQueSea (será unknown)

isContractCode(code: unknown): code is ContractApiErrorCode
    → ¿es un string Y está en el array apiErrorCodeValues?

isViolations(value: unknown): value is { [key: string]: string[] }
    → ¿es un objeto cuyos valores son TODOS arrays de strings?
```

`isContractCode` es el puente entre los dos mundos de la sección 1: el **tipo** `ContractApiErrorCode` no existe en runtime, así que se comprueba contra el **valor** `apiErrorCodeValues`, y el predicado le dice al compilador "a partir de aquí, trátalo como el tipo".

`isViolations` usa `.every(...)` dos veces: "cada valor del objeto es un array" y "cada elemento de ese array es string". `.every` devuelve `true` solo si la condición se cumple para todos.

Cómo se encadenan en toApiError (líneas 91-125)

Mira cómo el tipo de los datos se va estrechando paso a paso, desde "no sé nada" hasta "es un error del contrato":

```
const body: unknown = …                          // body: unknown
const error = isRecord(body) ? body.error        // body: Record<string, unknown>
                             : undefined         // error: unknown
if (isRecord(error) &&                           // error: Record<string, unknown>
    typeof error.message === 'string') {         // error.message: string
  if (isContractCode(error.code)) {              // error.code: ContractApiErrorCode
    if (error.code !== 'validation_failed') {    // error.code: los otros 9 códigos
      return new ApiError({ code: error.code, … })
    }
                                                 // aquí error.code: 'validation_failed'
    if (isViolations(error.violations)) {        // error.violations: { [k]: string[] }
      return new ApiErrorValidation(…)
    }
  }
  …              // código desconocido, o 'validation_failed' sin violations válidas
}                //   → 'unrecognized'
```

Fíjate en el orden: primero se pregunta "¿es un código del contrato?" y **solo dentro** se separa la validación. Tras el primer `return`, TypeScript sabe por descarte (sección 9) que `error.code` solo puede ser `'validation_failed'`. Y si ese código llega con `violations` ausentes o mal formadas, no se cuela como un `ApiError` normal: cae al final, a `'unrecognized'`, porque el back ha incumplido su propio contrato.

Cada `return new …` solo compila porque, en ese punto, TypeScript ya ha deducido que los datos tienen la forma que pide el constructor. Si borras una comprobación, el compilador se queja. Esa es la gracia: **la comprobación en runtime y el tipo en compilación quedan atados**.

11. Aserciones: as

`valor as Tipo` le dice al compilador "confía en mí, trátalo como este tipo". **No hace nada en runtime**: no convierte ni comprueba. Si te equivocas, nadie te avisa.

```
const input = document.getElementById('email') as HTMLInputElement
input.value       // ok para TS… pero si el elemento no existe, explota en runtime
```

Por eso hay que distinguir dos usos muy diferentes:

- **Ensanchar** (pasar a un tipo más general): seguro. Un `'a' | 'b'` siempre es un `string`.
- **Afirmar** algo que TS no puede saber: riesgo asumido. Debería estar justificado.

En client.ts

Línea 79 — ensanchar: `(apiErrorCodeValues as readonly string[]).includes(code)`.

`apiErrorCodeValues` está tipado como "array de solo lectura de los 10 códigos". Su `.includes()` exige que le pases *uno de esos 10*… pero justo queremos preguntar por un `string` cualquiera. Sin el `as`, TS da error. Tratar el array como `readonly string[]` es seguro (los 10 códigos son strings) y `readonly` respeta que no se puede modificar.

Línea 153 — afirmar: `(await response.json()) as { data: T }`.

`response.json()` devuelve `any`. Aquí se afirma que una respuesta correcta trae el sobre `{ data: … }`. No se comprueba en runtime porque el contrato del back lo garantiza para los 2xx; la desconfianza se concentra en las respuestas de error (comentario de cabecera, líneas 2-5).

Lo que el proyecto pide evitar

AGENTS.md pide no usar `any`, la aserción no-nula `!` (`valor!`, "te juro que no es null") ni `// @ts-ignore` para callar errores sin preguntar antes. `as` está en la misma familia: úsalo solo cuando sepas explicar por qué es correcto.

Parte IV · Tipos que fabrican tipos

12. Genéricos: <T>

Un **genérico** es un tipo-parámetro: un hueco que se rellena cuando usas la función. Igual que una función recibe valores como argumentos, una función genérica recibe además **tipos**.

```
function primero<T>(lista: T[]): T {
  return lista[0]
}

primero<number>([1, 2, 3])     // T = number → devuelve number
primero(['a', 'b'])            // T se deduce solo: string → devuelve string
```

Ya los has usado sin saberlo: `Array<string>` (= `string[]`), `Promise<Response>`... Son tipos genéricos de la librería estándar.

Restricciones: T extends X

`<T extends X>` limita qué tipos se aceptan en el hueco: "T puede ser cualquier cosa *siempre que* encaje en X".

```
function largo<T extends { length: number }>(x: T): number {
  return x.length
}
largo('hola')      // ok, string tiene length
largo([1, 2])      // ok
largo(42)          // ✗ number no tiene length
```

En client.ts

`export async function apiFetch<T>(path: string, options: RequestOptions = {}): Promise<T>`

`apiFetch` no sabe qué devuelve cada endpoint, así que deja un hueco `T` que rellena quien llama. Ejemplo real en `src/api/auth.ts`:

```
return apiFetch<RegisterUserResponse>('/auth/register', {
  method: 'POST',
  body: payload,
})
```

Aquí `T = RegisterUserResponse`, así que la función devuelve `Promise<RegisterUserResponse>` y el componente que la use tendrá autocompletado de los campos.

Honestidad: el genérico **no comprueba** nada en runtime; `T` solo viaja hasta el `as { data: T }` de la línea 153. Es quien llama quien declara qué espera recibir.

13. Utility types: Record, Pick, Omit, Extract

TypeScript trae "funciones de tipos" ya hechas, que reciben tipos y devuelven tipos nuevos. Son genéricos (sección 12) aplicados a transformar tipos.

Record<Claves, Valor>: objeto con claves de un tipo y valores de otro

```
type Notas = Record<string, number>          // { [clave: string]: number }
const n: Notas = { mates: 8, lengua: 6 }

type Stock = Record<'manzana' | 'pera', number>
const s: Stock = { manzana: 3, pera: 0 }      // ambas claves obligatorias
```

La forma `{ [key: string]: string[] }` que usa el contrato para `violations` es una **firma de índice** y significa lo mismo que `Record<string, string[]>`: "cualquier clave string, cada valor un array de strings". Por ejemplo `{ email: ['Ya existe'], password: ['Muy corta', 'Sin números'] }`.

Pick y Omit: quedarse con unas propiedades o quitar otras

```
interface Usuario { id: number; nombre: string; email: string; password: string }

type UsuarioPublico = Omit<Usuario, 'password'>       // id, nombre, email
type Credenciales   = Pick<Usuario, 'email' | 'password'>   // email, password
```

Extract y Exclude: filtrar miembros de una unión

```
type Letras = 'a' | 'b' | 'c'
type Comunes = Extract<Letras, 'a' | 'c' | 'z'>   // 'a' | 'c'  (los que están en ambas)
type Resto   = Exclude<Letras, 'a'>               // 'b' | 'c'  (quita los indicados)
type Nada    = Extract<Letras, 'x' | 'y'>         // never      (ninguno coincide)
```

En client.ts

Record (líneas 74-75): `isRecord` dice "es un objeto con claves string cuyos valores no conozco" → `Record<string, unknown>`. Así se puede escribir `body.error` sin que TS se queje, y lo que sale es `unknown` (hay que seguir comprobando).

Pick (línea 56):

```
interface ApiErrorValidationInit extends Pick<ApiErrorInit, 'message' | 'status'> {
  violations: ContractApiErrorValidation['violations']
}
```

De `ApiErrorInit` solo se toman `message` y `status`: `code` no, porque la subclase lo fija a `'validation_failed'`; `unrecognizedCode` tampoco, porque no aplica. Resultado: `{ message, status, violations }`. Y si mañana `message` cambia de tipo en `ApiErrorInit`, aquí cambia solo.

Omit (líneas 70-72):

```
interface RequestOptions extends Omit<RequestInit, 'body'> {
  body?: unknown
}
```

`RequestInit` es el tipo nativo del segundo argumento de `fetch` (`method`, `headers`, `signal`...). Su `body` solo acepta cosas ya serializadas (string, `FormData`, `Blob`...). Se quita ese `body` y se pone uno que acepta cualquier objeto, porque `apiFetch` lo convierte con `JSON.stringify`. Así quien llama escribe `body: payload` y no `body: JSON.stringify(payload)`.

Extract: lo usa el truco de la sección siguiente.

14. never y el truco AssertNoOverlap

never: el conjunto vacío

Piensa en un tipo como el **conjunto** de valores que admite: `boolean` = {true, false}, `'a' | 'b'` = {'a', 'b'}. `never` es el conjunto **vacío**: ningún valor encaja. Dos propiedades que vamos a usar:

- Unir con nada no añade nada: `X | never` es lo mismo que `X`.
- `Extract` devuelve `never` cuando no hay ningún miembro en común (ejemplo `Nada` de arriba).

Una restricción que solo acepta never

```
type SoloNever<T extends never> = T     // el hueco T solo admite 'never'

type Ok  = SoloNever<never>             // ok
type Mal = SoloNever<'hola'>            // ✗ Type '"hola"' does not satisfy the constraint 'never'
```

Es un tipo que no sirve para describir datos: sirve para **provocar un error de compilación** cuando algo que debería estar vacío no lo está. Es, en la práctica, un **test que se ejecuta en** `npm run type-check`.

En client.ts

```
type AssertNoOverlap<T extends never> = T

export type ApiErrorCode =
  | ContractApiErrorCode
  | FrontOnlyApiErrorCode
  | AssertNoOverlap<Extract<FrontOnlyApiErrorCode, ContractApiErrorCode>>
```

Léelo de dentro afuera:

1. `Extract<FrontOnlyApiErrorCode, ContractApiErrorCode>` = códigos que están *a la vez* en los del front y en los del back. Hoy ninguno → `never`.

2. `AssertNoOverlap<never>` → cumple la restricción → vale `never`.

3. `… | never` no añade nada a la unión: `ApiErrorCode` sigue siendo 10 + 2 códigos.

¿Y si un día el back añade un código llamado `unrecognized`? Tras regenerar los tipos, el paso 1 da `'unrecognized'`, que no es `never`, y `npm run type-check` falla en esta línea:

```
Type '"unrecognized"' does not satisfy the constraint 'never'.
```

Es lo que se quiere: en el back ese código tendría *otro significado*, y mezclarlo con el del front sería un bug silencioso. El error obliga a renombrar el código del front (comentario de las líneas 25-26).

¿Por qué meterlo dentro de la unión? Porque TypeScript solo comprueba un tipo si se usa en algún sitio. Ponerlo en `ApiErrorCode` garantiza que se evalúa, y como vale `never` no cambia nada.

Parte V · Recorrido completo de client.ts

Ya tienes todas las piezas. Ahora el archivo de arriba abajo, en bloques. Ábrelo al lado (`src/api/client.ts`) y ve siguiendo los números de línea.

15. El archivo de arriba abajo

Líneas	Qué hay	Conceptos (sección)
1-11	Comentario de cabecera: único sitio que llama a `fetch`; todo error acaba siendo `ApiError`.	—
12	Import de un valor (`apiErrorCodeValues`) y un tipo (`type components`).	Tipos vs valores (1)
14-17	Alias `Contract…` sacados del contrato generado.	`type` (2), acceso indexado (4)
19-23	Los dos códigos propios del front.	Literales y uniones (2)
25-32	`ApiErrorCode` = back + front, con el test de no-solapamiento.	Uniones (2), `Extract` (13), `never` (14)
34-39	Forma del objeto que recibe el constructor.	`interface`, `?` (3)
41-54	Clase `ApiError`: hereda de `Error`, guarda `code`, `status`, `unrecognizedCode`.	Clases, `readonly`, desestructurar parámetros (3, 5, 6)
56-68	Subclase de validación: añade `violations`, fija el código.	`Pick` (13), rest/spread (5), herencia (6)
70-72	Opciones de `apiFetch`: las de `fetch` con un `body` libre.	`Omit` (13), `unknown` (8)
74-89	Tres guardias: `isRecord`, `isContractCode`, `isViolations`.	Narrowing (9), predicados (10), `as` (11), `Record` (13)
91-125	`toApiError`: convierte cualquier respuesta de error en un `ApiError`.	`async` (7), `unknown` (8), narrowing en cadena (10)
127-155	`apiFetch`: construye la URL, llama a `fetch`, decide qué devolver o lanzar.	Genéricos (12), rest/spread (5), `try/catch` (7), `as` (11)

Detalles de apiFetch que aún no habíamos visto

- `import.meta.env.VITE_API_URL` (l.128): así lee Vite las variables de entorno. Solo se exponen al navegador las que empiezan por `VITE_`. Su valor sale de los ficheros `.env`.
- `new URL(path, base)` (l.128): construye una URL absoluta combinando la ruta (`'/auth/register'`) con la base del back.
- `body !== undefined ? JSON.stringify(body) : undefined` (l.141): un `GET` no lleva cuerpo; solo se serializa si hay algo que enviar.
- `'Accept-Language': 'es'` (l.138): el back usa esta cabecera para traducir los mensajes de `violations`. Está fijo hasta que el front tenga i18n.
- `response.ok` (l.149): `true` si el status HTTP está entre 200 y 299.

16. Los cinco finales posibles de apiFetch

Toda llamada a `apiFetch` termina exactamente de una de estas maneras. Quien llama solo tiene que preocuparse de dos cosas: el dato, o un `ApiError`.

#	Qué pasó	Resultado	Dónde
1	Sin respuesta (sin red, servidor caído, CORS): `fetch` lanza `TypeError`.	lanza `ApiError` con `code: 'network_error'`, `status: 0`	l.143-147
2	Error del back con sobre válido y un código del contrato distinto de `validation_failed`.	lanza `ApiError` con ese `code`	l.97-100
3	Error del back con sobre válido, `code: 'validation_failed'` y `violations` bien formadas.	lanza `ApiErrorValidation` (con `violations`)	l.102-108
4	Error con sobre pero código desconocido (el back desplegó uno nuevo y no se han regenerado tipos), o `validation_failed` con `violations` ausentes o mal formadas.	`console.error` + `ApiError` con `code: 'unrecognized'` y `unrecognizedCode` con el original	l.111-120
5	Error sin sobre (HTML de un proxy, cuerpo vacío, JSON raro).	`console.error` + `ApiError` `'unrecognized'` con `statusText` como mensaje	l.123-124
✓	Respuesta 2xx.	devuelve `data` (tipada como `T`)	l.153-154

Por qué tanta defensa solo aquí

El comentario de cabecera lo resume: el back garantiza su formato, pero lo que llega al navegador no siempre viene del back (proxies, cortes de red). En vez de repartir comprobaciones por todos los componentes, **toda la desconfianza vive en un único archivo** y el resto de la app trabaja con tipos fiables: o el dato, o un `ApiError` con un `code` de una lista cerrada.

Anexo A · Chuleta

Sintaxis	Significado	Línea
`x: T`	x es de tipo T	por todas partes
`import { type X }`	importa solo un tipo (se borra al compilar)	12
`type A = …`	alias: nombre para un tipo	15-23
`'a' | 'b'`	unión de literales: uno de estos valores	23
`T['k']`	tipo de la propiedad k de T	15-17, 36
`interface X { … }`	forma de un objeto	34
`p?: T`	propiedad opcional (T | undefined)	38, 45, 71
`readonly p`	no reasignable (solo compilador)	42-45, 61
`interface A extends B`	A tiene todo lo de B y más	56, 70
`class A extends B`	herencia; `super(…)` primero	41, 60
`{ a, ...resto } = obj`	desestructurar + recoger el resto	63, 129
`{ ...obj, k: v }`	copiar obj y sobrescribir k	64, 134-140
`(x = {})`	valor por defecto del parámetro	127
`unknown`	cualquier cosa, pero hay que comprobarla	71, 74-93
`x is T`	predicado: si devuelve true, x es T	74, 78, 82
`if (…) return` + código después	narrowing por descarte	98-102
`x as T`	aserción: "confía en mí" (sin efecto en runtime)	79, 153
`<T>`	genérico: hueco de tipo	127
`<T extends X>`	genérico restringido	27
`Promise<T>`	valor T que llegará más tarde	91, 127
`Record<K, V>`	objeto con claves K y valores V	74
`Pick<T, 'a'>` / `Omit<T, 'a'>`	quedarse con / quitar propiedades	56 / 70
`Extract<A, B>`	miembros de A que también están en B	32
`never`	tipo vacío; `X | never` = X	27, 32

Anexo B · Ejercicios

Hazlos en un archivo de prueba suelto (o en el Playground oficial: typescriptlang.org/play). Las soluciones están en la página siguiente.

Ejercicio 1 · Uniones

Define un tipo `Talla` que solo admita `'S'`, `'M'` y `'L'`. ¿Qué error da `const t: Talla = 'XL'`?

Ejercicio 2 · Acceso indexado

Dado `interface Libro { titulo: string; autor: { nombre: string; pais: string } }`, saca en un tipo `Pais` el tipo de `pais` sin escribir `string`.

Ejercicio 3 · Rest y spread

¿Qué valen `a` y `resto`? `const { a, ...resto } = { a: 1, b: 2, c: 3 }`. Y ¿qué vale `{ ...{ x: 1, y: 2 }, y: 9 }`?

Ejercicio 4 · Predicado

Escribe `esNumeroPositivo(x: unknown): x is number` que devuelva `true` solo para números mayores que 0. Úsala en un `if` y llama a `x.toFixed(1)` dentro.

Ejercicio 5 · Omit

Con `interface Usuario { id: number; nombre: string; password: string }`, crea `NuevoUsuario` (sin `id`) y `UsuarioPublico` (sin `password`).

Ejercicio 6 · Lectura de client.ts

Si el back responde `500` con el cuerpo `<html>Bad Gateway</html>`, ¿qué final de la tabla de la sección 16 ocurre? ¿Qué valen `code` y `message` del error?

Ejercicio 7 · El truco

Imagina que añades `'http_error'` a `FrontOnlyApiErrorCode`. ¿Qué pasa al ejecutar `npm run type-check` y por qué? (Pista: mira la lista de códigos del back en la sección 4.)

Ejercicio 8 · Validación sin violations

El back responde `422` con `{ "error": { "code": "validation_failed", "message": "Datos inválidos" } }`, sin `violations`. ¿Qué final de la sección 16 ocurre? ¿Por qué no se devuelve un `ApiError` normal con `code: 'validation_failed'`?

Anexo C · Soluciones

1

```
type Talla = 'S' | 'M' | 'L'
const t: Talla = 'XL'   // ✗ Type '"XL"' is not assignable to type 'Talla'.
```

2

```
type Pais = Libro['autor']['pais']   // string
```

3

```
a = 1, resto = { b: 2, c: 3 }
{ ...{ x: 1, y: 2 }, y: 9 }  →  { x: 1, y: 9 }   // lo de después gana
```

4

```
function esNumeroPositivo(x: unknown): x is number {
  return typeof x === 'number' && x > 0
}
const v: unknown = 3.14159
if (esNumeroPositivo(v)) {
  v.toFixed(1)   // '3.1'
}
```

5

```
type NuevoUsuario   = Omit<Usuario, 'id'>         // { nombre, password }
type UsuarioPublico = Omit<Usuario, 'password'>   // { id, nombre }
```

6

`response.json()` falla (no es JSON) → `.catch(() => null)` → `body = null` → `isRecord(body)` es `false` → `error = undefined` → no entra en el `if` → **final 5**: `console.error` y `ApiError` con `code: 'unrecognized'`, `status: 500` y `message` = `response.statusText` (normalmente `'Internal Server Error'`).

7

`'http_error'` **ya es un código del back**. `Extract<FrontOnlyApiErrorCode, ContractApiErrorCode>` da `'http_error'` en vez de `never`, no cumple `T extends never` y el type-check falla con `Type '"http_error"' does not satisfy the constraint 'never'`. El truco está haciendo su trabajo: evita que el front reutilice un nombre que en el back significa otra cosa.

8

`isContractCode` da `true`, pero el código es `'validation_failed'`, así que no entra en el primer `return`. Después `isViolations(undefined)` da `false` (no es un objeto), se sale del bloque y ocurre el **final 4**: `console.error` y `ApiError` con `code: 'unrecognized'`, `unrecognizedCode: 'validation_failed'` y `status: 422`. No se devuelve un `ApiError` con `'validation_failed'` porque quien captura ese código espera poder leer `violations` (un `ApiErrorValidation`); mejor marcarlo como respuesta no reconocida que dar un error a medias.